TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

ĐẠI HỌC BÁCH KHOA HÀ NỘI

BÁO CÁO BÀI TẬP LỚN

GVHD: BÙI THỊ MAI ANH

TOPIC 8: SINGLE SOURCE SHORTEST PATH(SSSP)

THÀNH VIÊN:

LÊ DƯỢC HOÀNG – 20194573
NGUYỄN HOÀI NAM – 20194628
HÀ THỊ LINH – 20194603

HÀ NỘI, NGÀY 13 THÁNG 7 NĂM 2022

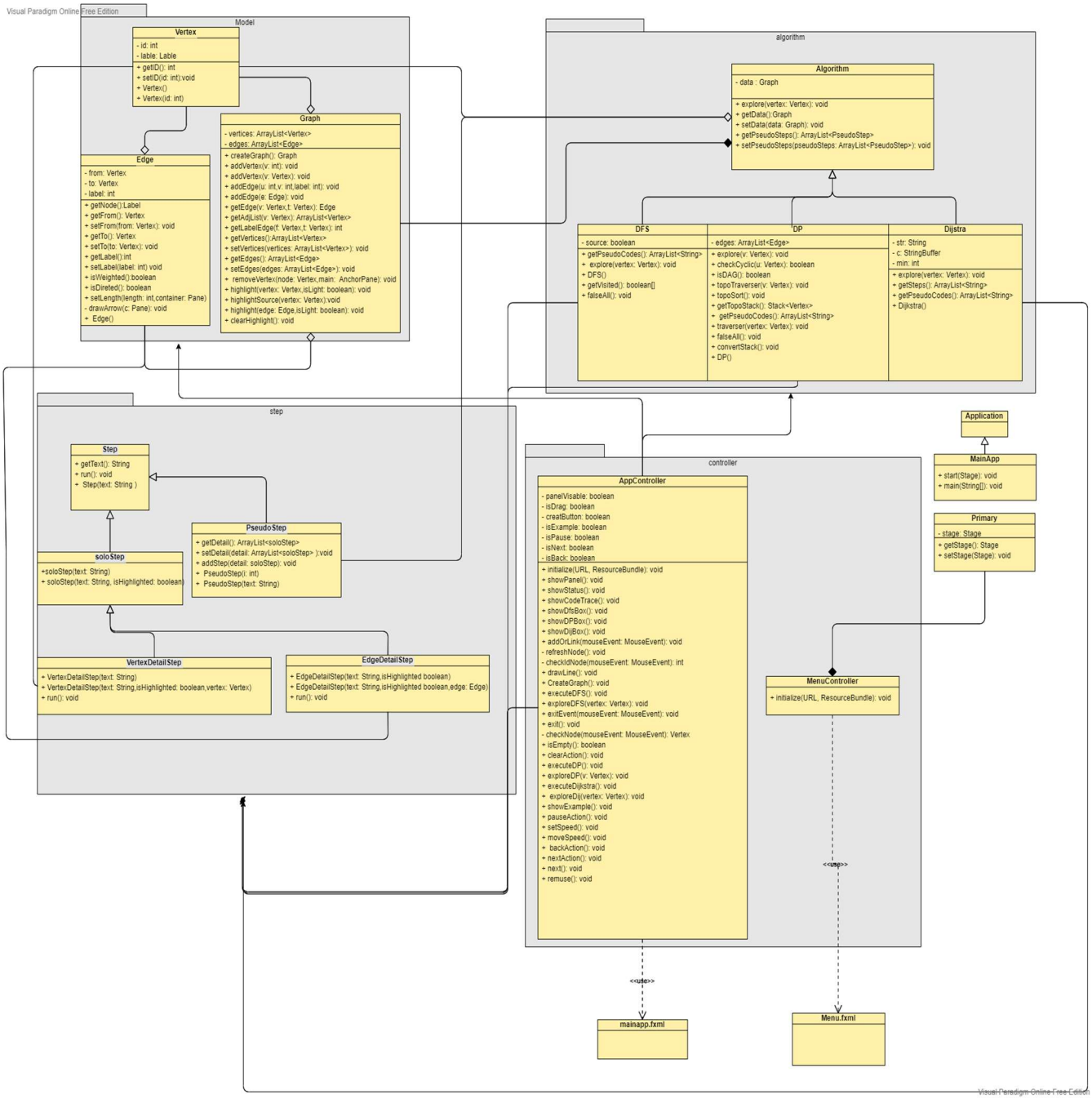
Đề tài: Cho một đồ thị G có hướng và có trọng số. Bài toán đặt ra là tìm đường đi ngắn nhất từ một đỉnh đến các đỉnh còn lại. Sử dụng các thuật toán như sau:

- Dynamic Programming
- DFS
- Dijkstra

Yêu cầu: Sử dụng Java và Javafx
Giới thiệu về các thuật toán:

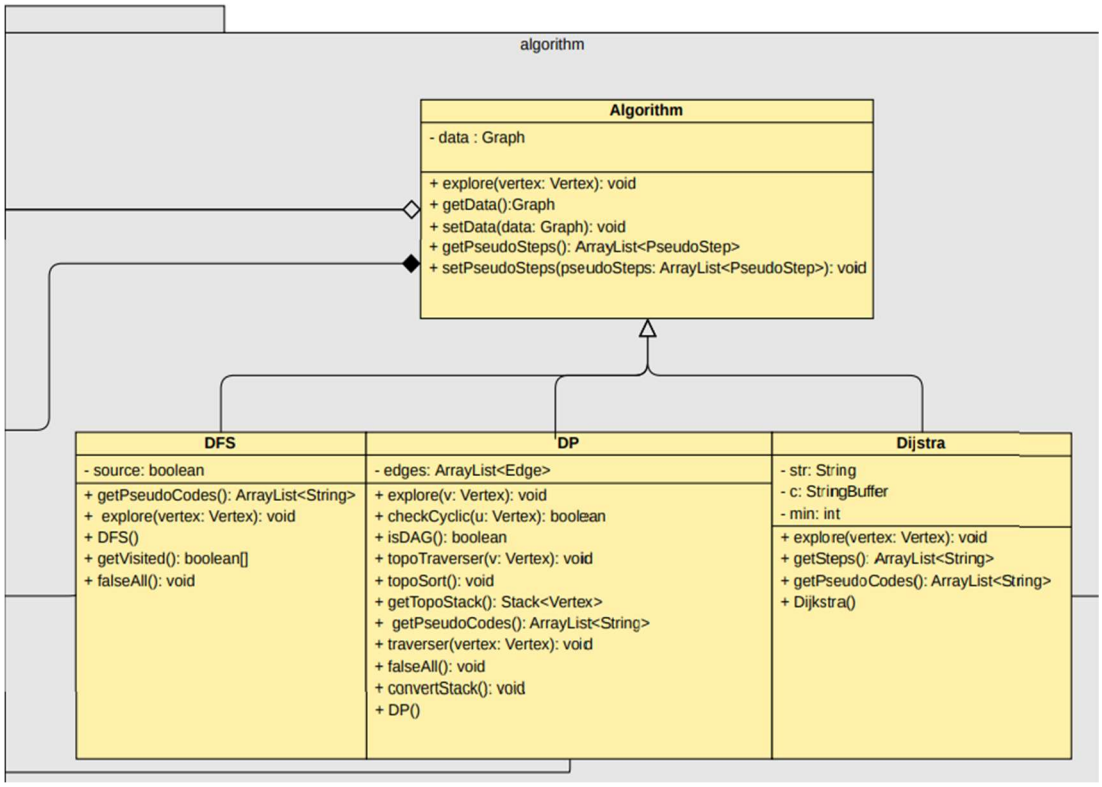
- **Dynamic Programming**(quy hoạch động) là một kỹ thuật trong lập trình giúp giải quyết một cách hiệu quả một lớp vấn đề có các bài toán con chồng chéo và thuộc tính cấu trúc con tối ưu. Những bài toán như vậy liên quan đến việc tính toán nhiều lần giá trị của các bài toán con giống nhau để tìm ra giải pháp tối ưu.
- DFS (**Depth First Search**) : còn được gọi là giải thuật tìm kiếm ưu tiên chiều sâu, là giải thuật duyệt hoặc tìm kiếm trên một cây hoặc một đồ thị và sử dụng stack (ngăn xếp) để ghi nhớ đỉnh liền kề để bắt đầu việc tìm kiếm khi không gặp được đỉnh liền kề trong bất kỳ vòng lặp nào. Giải thuật tiếp tục cho tới khi gặp được đỉnh cần tìm hoặc tới một nút không có con. Khi đó giải thuật quay lui về đỉnh vừa mới tìm kiếm ở bước trước.
- **Dijkstra** : thuật toán sẽ duy trì đường đi ngắn nhất đến tất cả các đỉnh. Ở mỗi bước, chọn đường đi S→u có tổng trọng số nhỏ nhất trong tất cả các đường đi đang được duy trì. Sau đó tiến hành tối ưu các đường đi S→v bằng cách thử kéo dài thành S→u→v như đã mô tả trên.

- Đồ thị UML

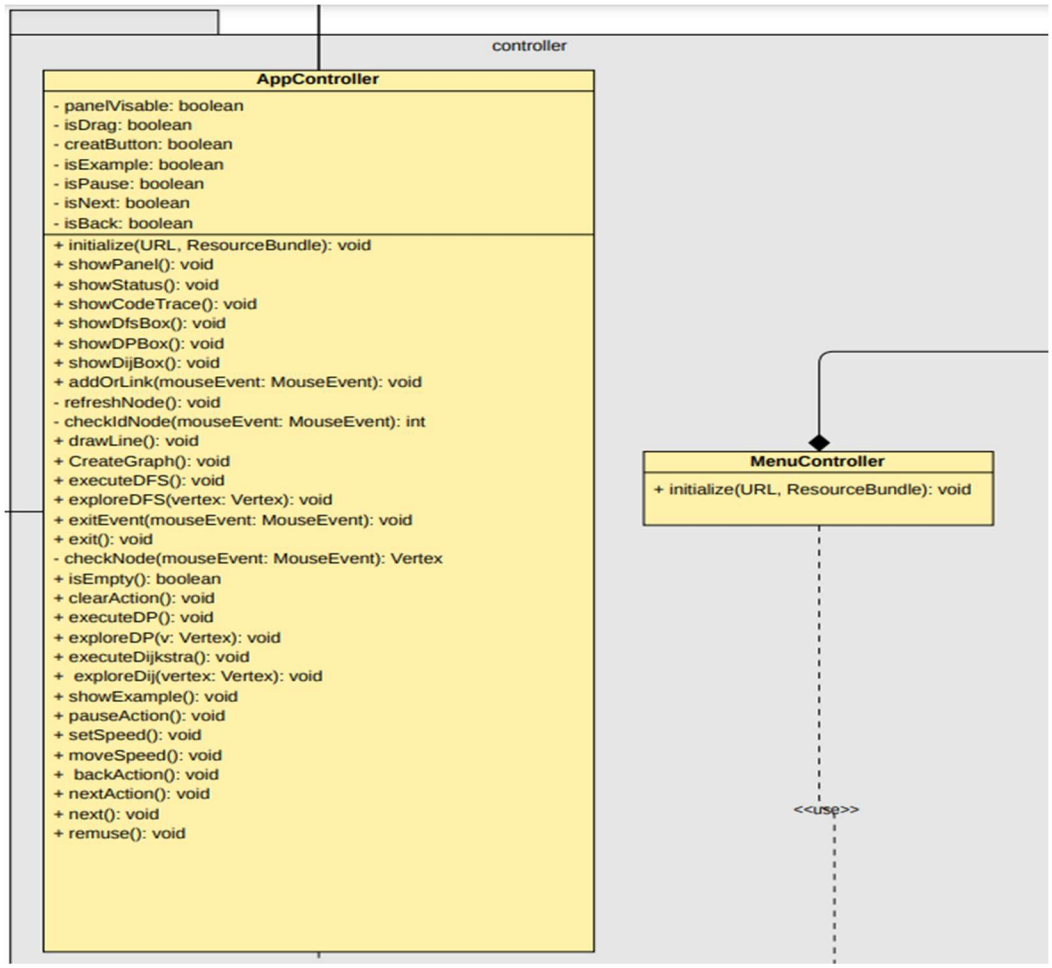


- DESIGN CLASS DIAGRAM

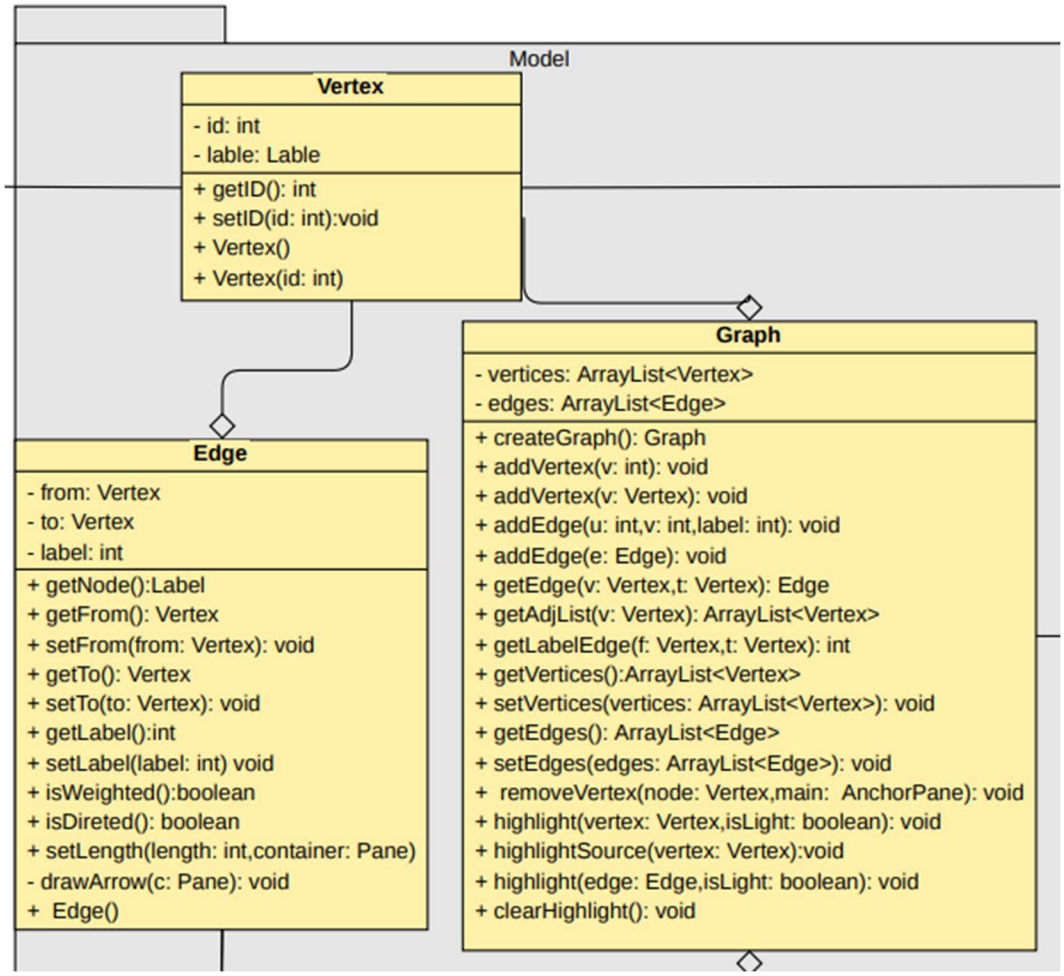
1. algorithm



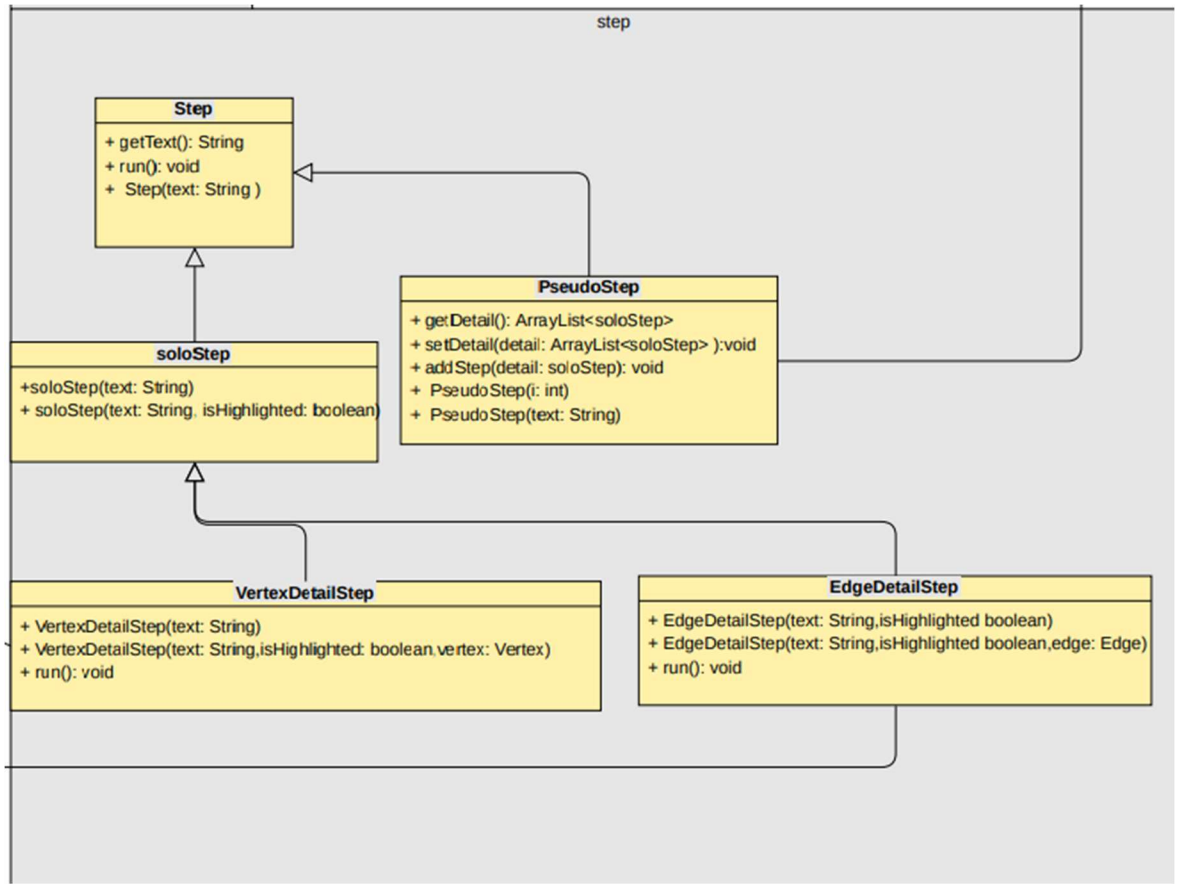
2. controller


controller
AppController
- panelVisable: boolean
- isDrag: boolean
- creatButton: boolean
- isExample: boolean
- isPause: boolean
- isNext: boolean
- isBack: boolean
+ initialize(URL, ResourceBundle): void
+ showPanel(): void
+ showStatus(): void
+ showCodeTrace(): void
+ showDfsBox(): void
+ showDPBox(): void
+ showDijBox(): void
+ addOrLink(mouseEvent: MouseEvent): void
- refreshNode(): void
- checkIdNode(mouseEvent: MouseEvent): int
+ drawLine(): void
+ CreateGraph(): void
+ executeDFS(): void
+ exploreDFS(vertex: Vertex): void
+ exitEvent(mouseEvent: MouseEvent): void
+ exit(): void
- checkNode(mouseEvent: MouseEvent): Vertex
+ isEmpty(): boolean
+ clearAction(): void
+ executeDP(): void
+ exploreDP(v: Vertex): void
+ executeDijkstra(): void
+ exploreDij(vertex: Vertex): void
+ showExample(): void
+ pauseAction(): void
+ setSpeed(): void
+ moveSpeed(): void
+ backAction(): void
+ nextAction(): void
+ next(): void
+ remuse(): void
MenuController
+ initialize(URL, ResourceBundle): void
<<use>>

3. model


Model
Vertex
- id: int
- lable: Lable
+ getID(): int
+ setID(id: int):void
+ Vertex()
+ Vertex(id: int)
Edge
- from: Vertex
- to: Vertex
- label: int
+ getNode():Label
+ getFrom(): Vertex
+ setFrom(from: Vertex): void
+ getTo(): Vertex
+ setTo(to: Vertex): void
+ getLabel():int
+ setLabel(label: int) void
+ isWeighted():boolean
+ isDireted(): boolean
+ setLength(length: int,container: Pane)
- drawArrow(c: Pane): void
+ Edge()
Graph
- vertices: ArrayList<Vertex>
- edges: ArrayList<Edge>
+ createGraph(): Graph
+ addVertex(v: int): void
+ addVertex(v: Vertex): void
+ addEdge(u: int,v: int,label: int): void
+ addEdge(e: Edge): void
+ getEdge(v: Vertex,t: Vertex): Edge
+ getAdjList(v: Vertex): ArrayList<Vertex>
+ getLabelEdge(f: Vertex,t: Vertex): int
+ getVertices():ArrayList<Vertex>
+ setVertices(vertices: ArrayList<Vertex>): void
+ getEdges(): ArrayList<Edge>
+ setEdges(edges: ArrayList<Edge>): void
+ removeVertex(node: Vertex,main: AnchorPane): void
+ highlight(vertex: Vertex,isLight: boolean): void
+ highlightSource(vertex: Vertex):void
+ highlight(edge: Edge,isLight: boolean): void
+ clearHighlight(): void

4. step



- Các packages:

Com.example.oop:
+)Com.example.oop.algorithm:
Algorithm.java:Lớp trìu tượng chứa dữ liệu của đồ thị và các phương trình trìu tượng để lớp con override lại.
DFS.java:Chạy thuật toán DFS.
Dijkstra.java:Chạy thuật toán Dijkstra.
DP.java:Chạy thuật toán Dynamic Programming.

+)Com.example.oop.controller:
AppController.java:Điều khiển các chức năng của Application.
MenuController.java:Khởi tạo Menu.

+) Com.example.oop.model:
Vertex.java:Chứa Id của đỉnh
Edge.java:Chứa cung giữa và đỉnh và trọng số.
Graph.java:Các phương thức thêm đỉnh và nối cung , xoá đỉnh , xoá cung ,

highlight đỉnh,highlight cung.

+)Com.example.oop:
Step.java:Lớp trìu tượng chứa biến step(String).
SoloStep.java:Lớp con kế thừa lớp Step,lưu detail step và highlight step.
PseudoStep.java:Lớp con kế thừa lớp Step,lưu 1 danh sách các detail step.
VertexDetailStep.java:Lớp con kế thừa soloStep,lưu detail step của đỉnh và highlight đỉnh.
EdgeDetailStep.java:Lớp con kế thừa soloStep,lưu detail step của cung và highlight cung.

- Các tính chất:

+ Tính đóng gói:Các biến instance của các class trong project được để private chỉ sử dụng phương thức setter và getter để thiết lập và lấy dữ liệu từ các biến.

+ Tính kế thừa :

- Có hai Abstract Class đó là Algorithm và Step.Các lớp con kế thừa có thể sử dụng lại các thuộc tính và phương thức của lớp cha.

+)Ví dụ:

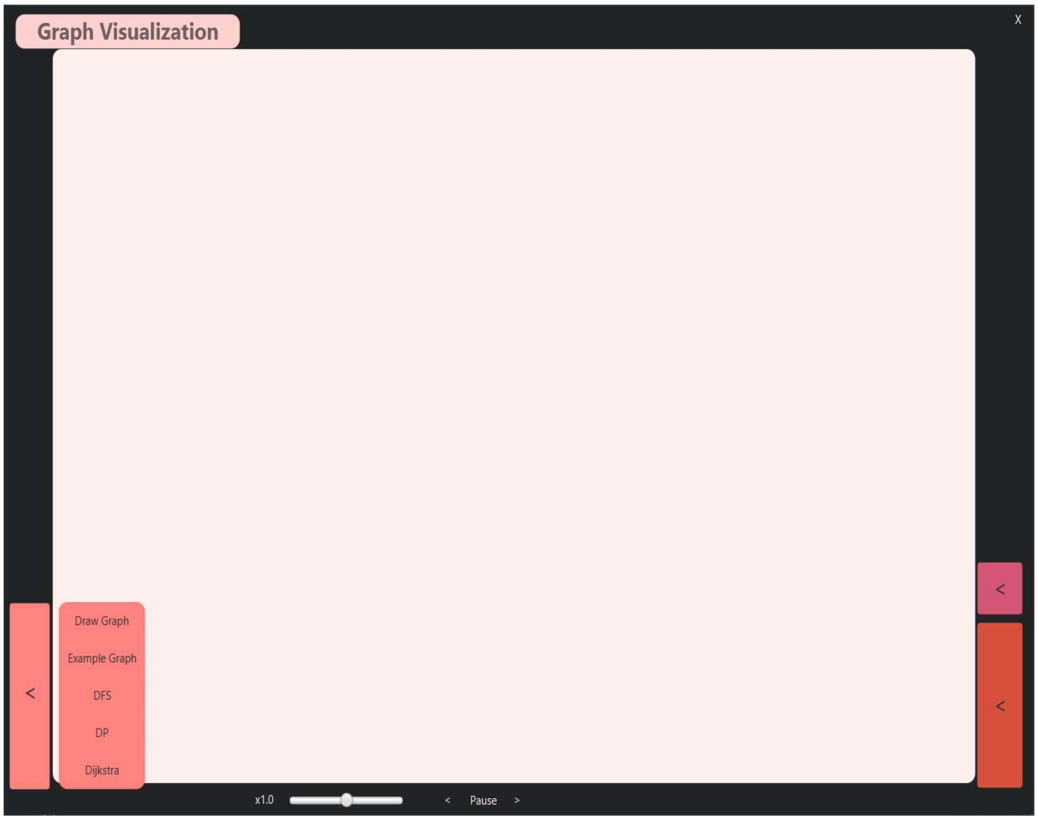
- Các lớp con DFS, Dijkstra,DP sẽ sử dụng phương thức setData của lớp cha (Algorithm) để nạp dữ liệu từ đồ thị.
- Các lớp con PseudoStep và SoloStep sẽ sử dụng lại constructor của lớp cha để khởi tạo public Step(String text)

+ Tính đa hình:

- Nạp chồng các phương thức Static trong class
  Ví dụ:Trong class Graph có 2 phương thức nạp chồng là public void addEdge(int u,int v,int weight) và public void addEdge(Edge e)
- Sử dụng lại phương thức của lớp cha tuỳ vào mục đích của Class
  Ví dụ: Class DFS,Dijkstra,DP sử dụng lại Abtract method explore(vertex v) để thiết lập thuật toán riêng cho các Class đó.

- Test :

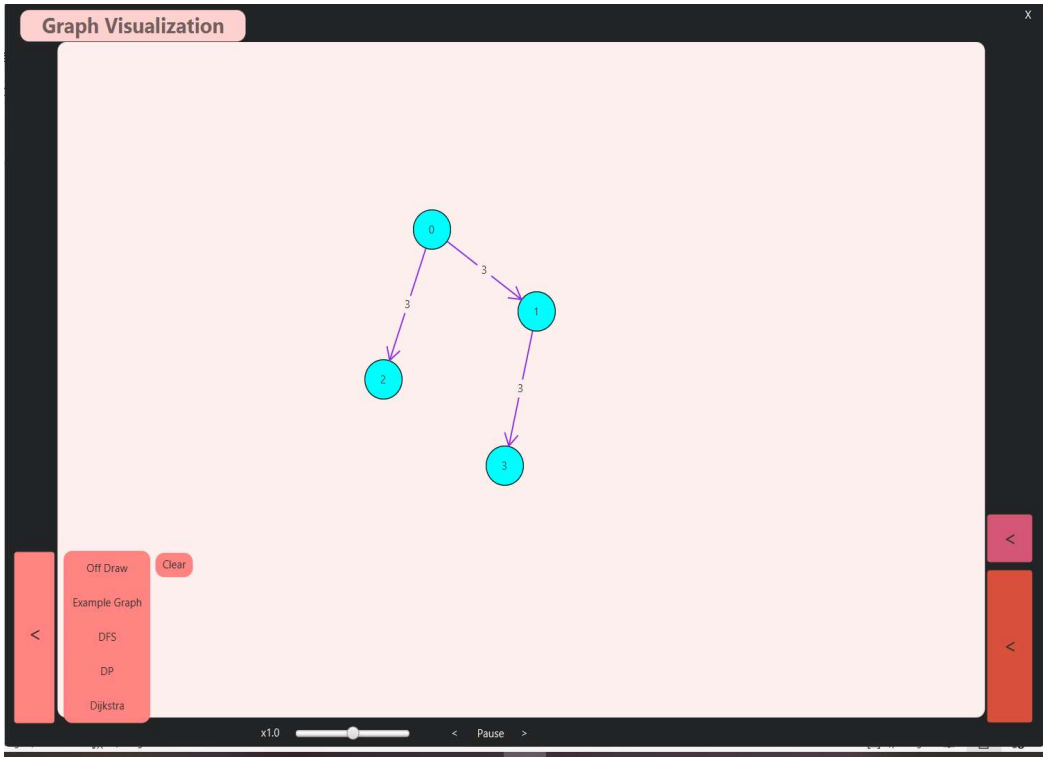
+ giao diện khi tổng quan:



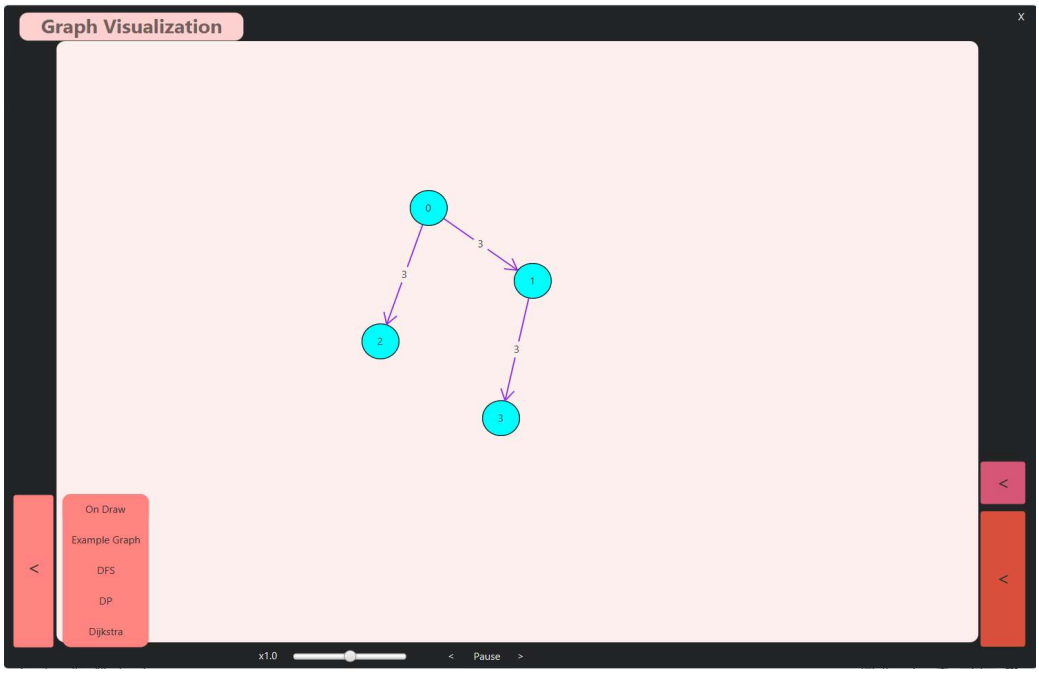
+ Vẽ đồ thị:

- B1: click chuột vào “Draw Graph”

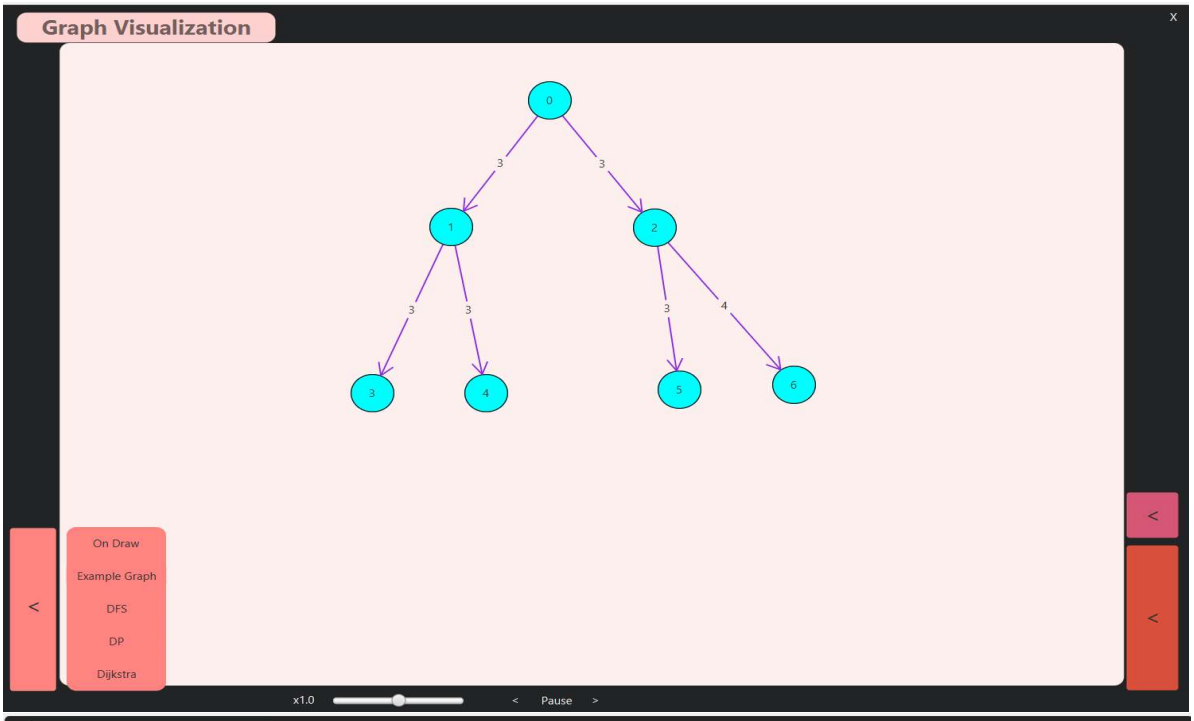
- sau đó ấn vào vùng trống để tạo Graph

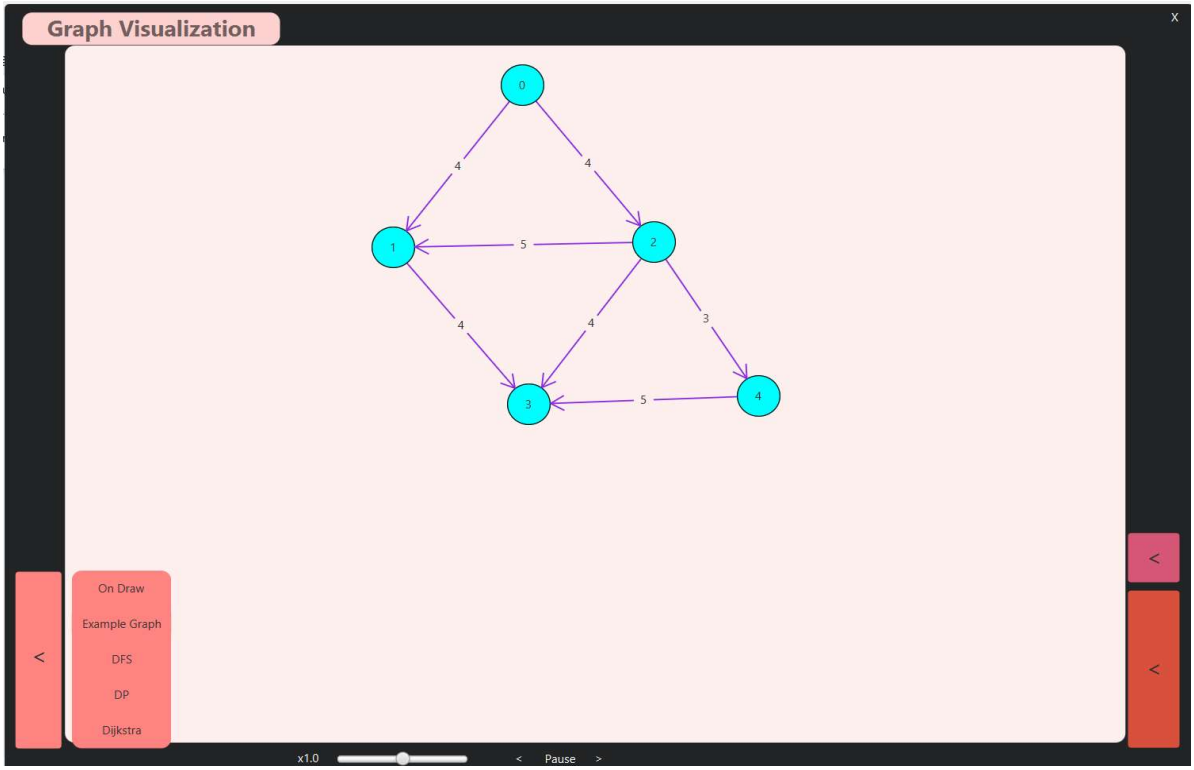


- nếu muốn dừng vẽ thì nhấn lại vào offDraw, còn muốn xóa graph vừa vẽ thì nhấn vào clear



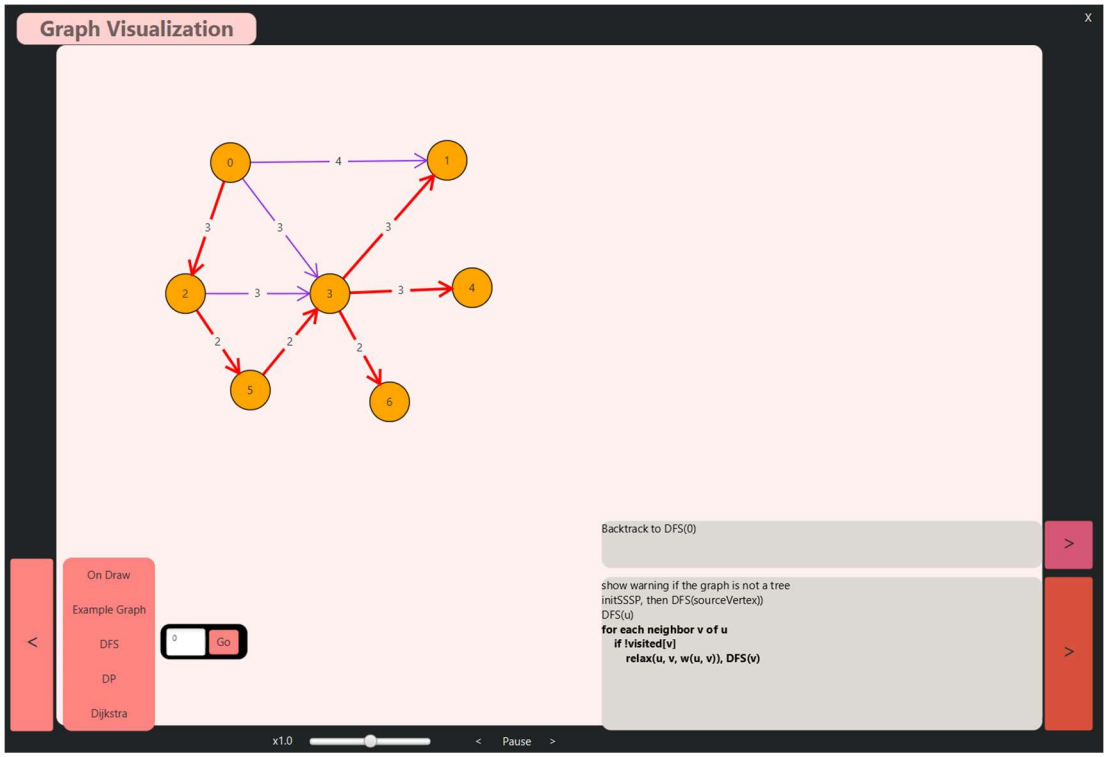
+ Muốn chọn đồ thị sẵn có thì nhấn vào “Example Graph” thì mỗi lần sẽ ra random 1 đồ thị khác nhau:



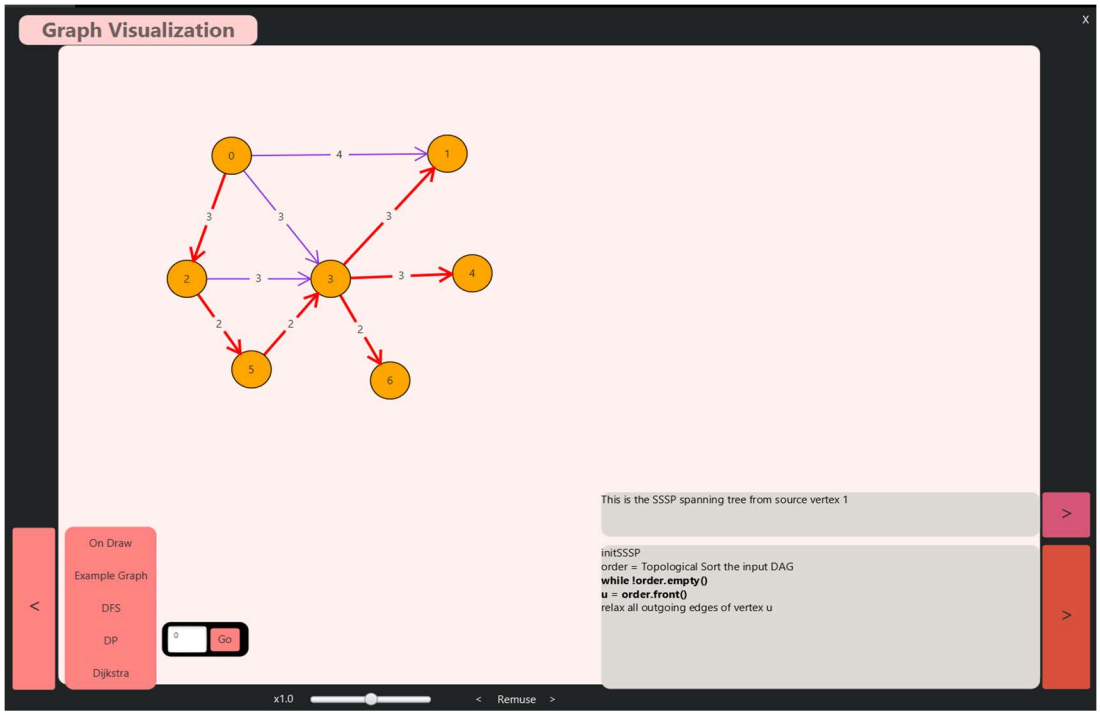


+ Khởi chạy thuật toán:

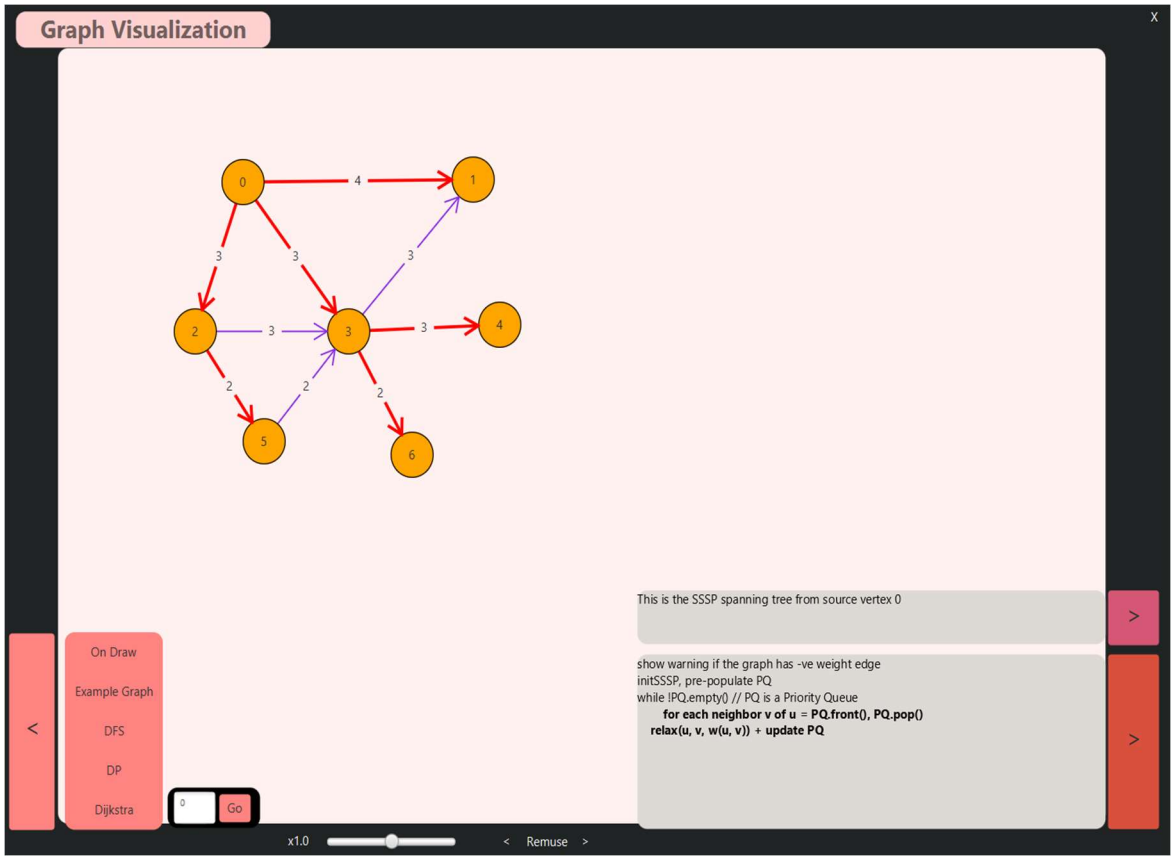
B1: chọn thuật toán mình muốn chạy
B2: Sau đó nhập đỉnh bắt đầu, rồi ấn Go:
+ DFS:



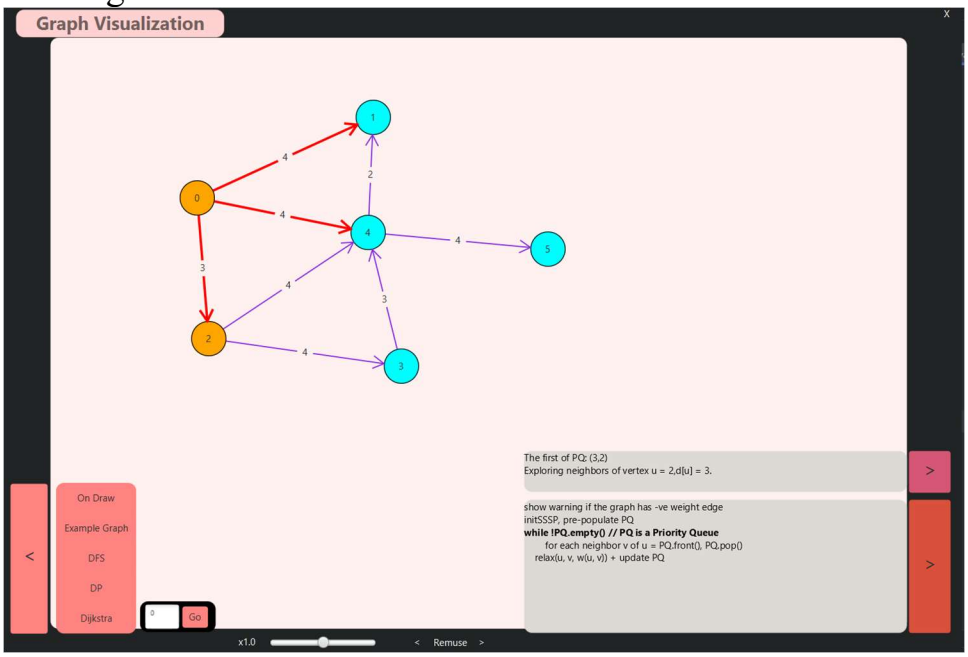
+ DP:


Graph Visualization
x
0
1
2
3
4
5
6
4
3
3
3
3
3
2
2
2
This is the SSSP spanning tree from source vertex 1
initSSSP
order = Topological Sort the input DAG
while !order.empty()
u = order.front()
relax all outgoing edges of vertex u
On Draw
Example Graph
DFS
DP
Dijkstra
0
Go
x1.0
Remuse

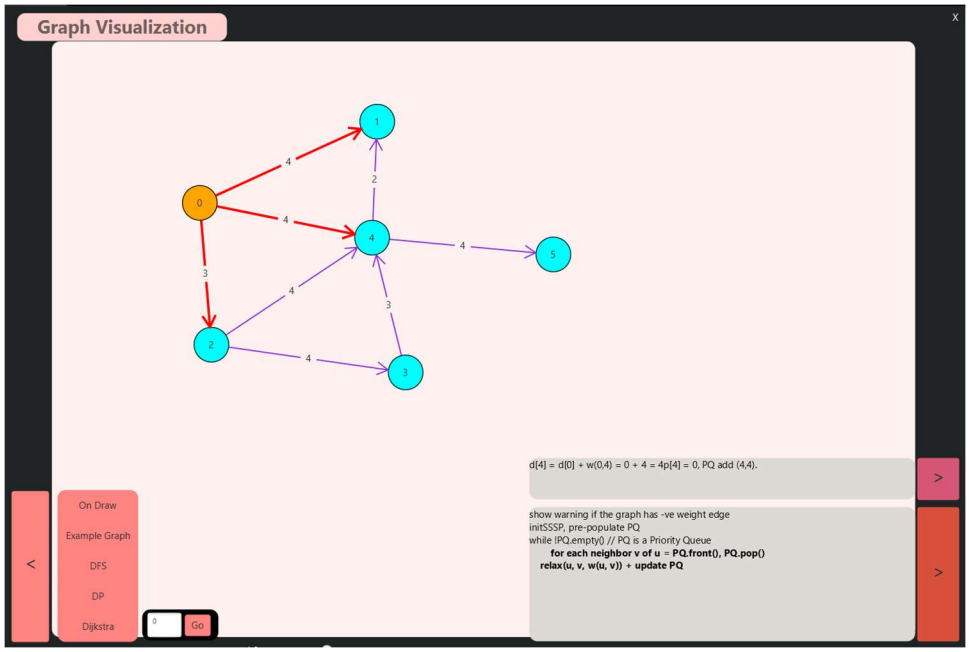
+ dijkstra



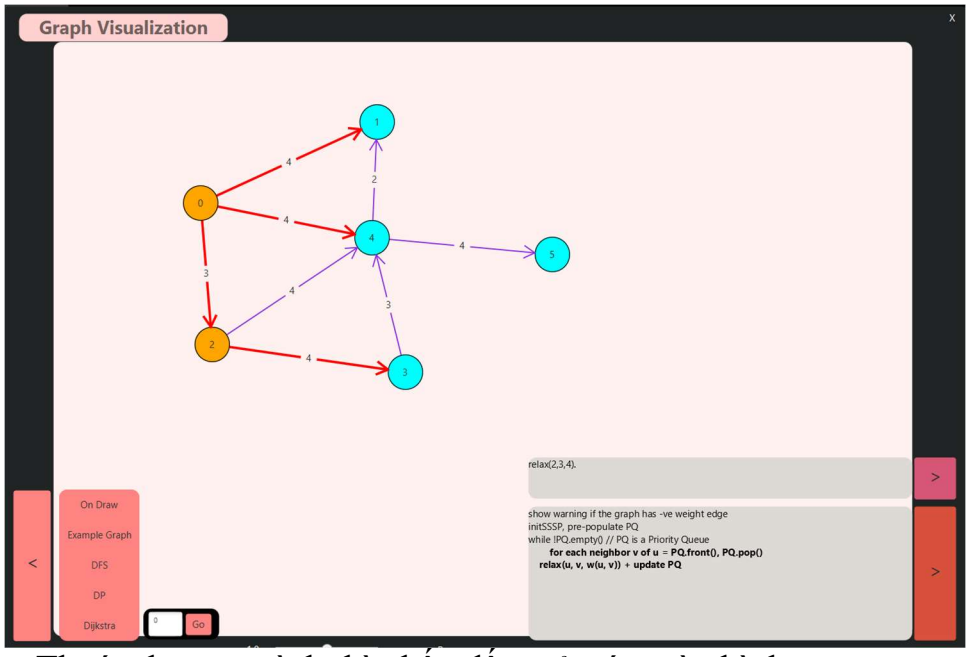
+ có thêm chức năng back và next



- back



- next:



+ Thoát chương trình thì nhấn dấu x ở góc màn hình